

Lectures on Neural Networks

Pedro Tarancón-Álvarez
Pablo Tejerina-Pérez

November 17, 2025

Abstract

Contents

1	Introduction	2
2	Neural Networks	2
2.1	Brief history of Machine Learning	2
2.2	Definition of Neural networks	3
2.2.1	Single neurons (perceptron): weights, bias, activation function	3
2.2.2	Architecture: multiple neurons, layers and connections	4
2.3	Learning process	6
2.3.1	Loss function: general case	6
2.3.2	Optimization process: Stochastic Gradient Descent (SGD)	8
2.3.3	Adam optimizer	9
2.3.4	Overfitting	10
3	PINNs	10
3.1	Definition	10
3.2	Loss function from the Differential Equations	11
3.3	Initial or Boundary Conditions (IC/BC)	11
3.3.1	Imposition on the loss function (soft enforcing)	12
3.3.2	Imposition by construction in the architecture of the net (hard enforcing)	13
3.4	Bundle solutions	13
3.5	Advantages and disadvantages	14
4	Dimensionality reduction techniques	15
4.1	Principal component analysis	15
4.2	t-distributed stochastic neighbor embedding	16
4.3	Uniform manifold approximation and projection	17
4.4	Machine learning approach: autoencoders	18
5	Hands-on sessions	19
5.1	Solving the Forced Harmonic Oscillator (FHO) and the Van der Pol Oscillator (VdPO) in a bundle using PINNs	19
5.2	Dimensionality reduction with NNs	20

1 Introduction

These notes are thought to be taken as a summary of the second part of the subject “Mathematical and Statistical Techniques”. In particular, they will focus on practical aspects of Machine Learning applied to different cases. We will first start with an introduction to Neural Networks, the main object that we are going to use during the lessons. Then, we will explain Physics Informed Neural Network, a novel way of using these objects to solve complex differential equations. We will also remark the main problems that PINNs have when compared to traditional numerical methods. Then, we will present how Neural Networks can be used to perform other tasks, as learning clustering of data or performing a dimensionality reduction. We will compare these techniques with non-machine learning ones like Principal Component Analysis (PCA) or UMAP. At last, we will focus on a couple of hands-on sessions. The aim of these is to illustrate the concepts explain during the theory classes.

2 Neural Networks

2.1 Brief history of Machine Learning

The history of machine learning (ML) and neural networks (NNs) extends across nearly a century of developments linking neuroscience, mathematics, and computer science. The origins can be traced back to the 1940s, when Warren McCulloch and Walter Pitts proposed a logical model of the neuron in their 1943 paper “*A Logical Calculus of Ideas Immanent in Nervous Activity*”. Their artificial neurons could perform logical operations, marking the conceptual birth of neural computation. Around the same time, Donald Hebb (1949) formulated his famous rule “neurons that fire together wire together,” providing the first theory of synaptic learning and inspiring later unsupervised algorithms. Alan Turing (1950) speculated about machines that could learn from experience, while Arthur Samuel (1952–1957) built a checkers-playing program that improved through self-play, coining the term *machine learning*.

In 1957, Frank Rosenblatt introduced the *Perceptron*, a computational model inspired by biological neurons and capable of learning simple patterns. The perceptron generated enormous optimism about artificial intelligence, even leading to hardware implementations such as the Mark I Perceptron. However, this enthusiasm was short-lived: in 1969, Marvin Minsky and Seymour Papert published “*Perceptrons*”, showing that single-layer perceptrons could not represent basic non-linear functions like XOR. This mathematical limitation triggered the first *AI winter*, during which research on neural networks largely ceased.

During the 1970s, symbolic artificial intelligence and expert systems dominated. Researchers focused on rule-based reasoning rather than data-driven learning. Nevertheless, a few important neural advances occurred beneath the surface: Teuvo Kohonen developed self-organizing maps (1972), and Paul Werbos (1974) formulated the backpropagation algorithm, though it went mostly unnoticed at the time.

In the 1980s, neural networks experienced a major revival. In 1986, David Rumelhart, Geoffrey Hinton, and Ronald Williams rediscovered and popularized *backpropagation*, enabling the training of multilayer perceptrons (MLPs). Around the same time, John Hopfield (1982) introduced recurrent networks with attractor dynamics, while Hinton and Sejnowski (1985) developed Boltzmann Machines, linking statistical physics and neural computation. These ideas reestablished neural networks as a serious learning framework, leading to practical successes in

character and speech recognition.

The 1990s marked the rise of statistical machine learning, characterized by theoretical rigor and probabilistic reasoning. Researchers such as Vladimir Vapnik developed *Support Vector Machines* (SVMs), and statistical learning theory formalized notions like bias–variance tradeoff and VC dimension. Other methods, such as decision trees, ensembles, and probabilistic graphical models, flourished. During this period, shallow neural networks declined in popularity, but the mathematical foundations for modern learning were solidified.

The 2000s and 2010s witnessed the *deep learning revolution*. In 2006, Hinton and Salakhutdinov demonstrated that deep architectures could be effectively trained using layer-wise pre-training of *Deep Belief Networks*. With the rise of GPUs and large labeled datasets such as ImageNet, training large-scale neural networks became feasible. A decisive breakthrough came in 2012 with **AlexNet** (Krizhevsky, Sutskever, Hinton), which dominated the ImageNet competition and demonstrated the power of deep convolutional networks. Shortly after, major innovations followed: **GANs** (Goodfellow, 2014) introduced adversarial training, **ResNets** enabled extremely deep architectures, and **AlphaGo** and Transformer-based models transformed decision-making and natural language processing.

In the 2020s, the field entered the era of *foundation models*. The Transformer architecture (Vaswani et al., 2017) made it possible to capture long-range dependencies and train models at unprecedented scale. Systems such as BERT, GPT, T5, CLIP, and diffusion models extended deep learning across text, images, and multimodal tasks. Modern machine learning now integrates ideas from probabilistic reasoning, reinforcement learning, and even physics-informed neural networks (PINNs), reflecting a unification of methods across scientific disciplines.

The historical trajectory of machine learning and neural networks thus mirrors a recurring cycle of optimism, decline, and resurgence—driven by progress in theory, data availability, and computational power. From the logical neurons of McCulloch and Pitts to today’s foundation models with billions of parameters, the field has evolved into a central pillar of artificial intelligence, bridging mathematics, physics, and cognitive science.

2.2 Definition of Neural networks

An artificial neural network (ANN) is a machine learning algorithm that uses a structure of interconnected nodes, disposed in a layered structure, to solve a problem through the minimization of some error function. To understand how this can be done, we need to first understand the behavior of the building blocks of a neural network (NN): the nodes, usually called artificial neurons or just neurons.

2.2.1 Single neurons (perceptron): weights, bias, activation function

Artificial neurons are inspired in neuron cells of our brains. A single neuron can be seen as a “choice-making device” [1]. Specifically, it takes n inputs in the form of a vector:

$$\vec{x} = (x_1, \dots, x_n),$$

and produces a single output y , usually between -1 and 1 . The inputs x_i can be thought of as the “factors accounted for in the choice-making process”, and the output y as the “choice made by the neuron”. Each input has an associated *weight* w_i (“relevance given to each input factor”), and an overall *bias* b , which controls how easily the neuron gives an output close to 1

(“how easily it *fires*”). Last, a single neuron has an *activation function* $f(x; w, b)$ which models its response¹. A single neuron is depicted in figure 1.

One of the most common activation functions is the hyperbolic tangent, which reminds us of a smooth step function (fig. 2a):

$$f(x; w, b) = \tanh \left(\sum_{i=1}^n w_i x_i + b \right). \quad (1)$$

There are other activation functions that are not bounded from above, such as the *Sigmoid Linear Units* function (SiLU) (fig. 2b). This function allows for NNs to search for more “exotic behaviors”.

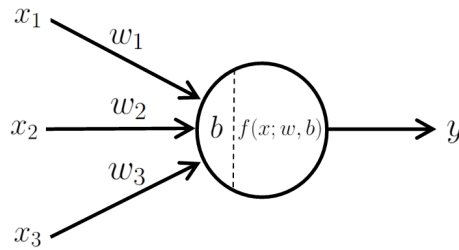


Figure 1: A single neuron with 3 inputs (x_i) and a single output (y). Each input has a corresponding weight (w_i). The neuron has also a bias b and an activation function $f(x; w, b)$.

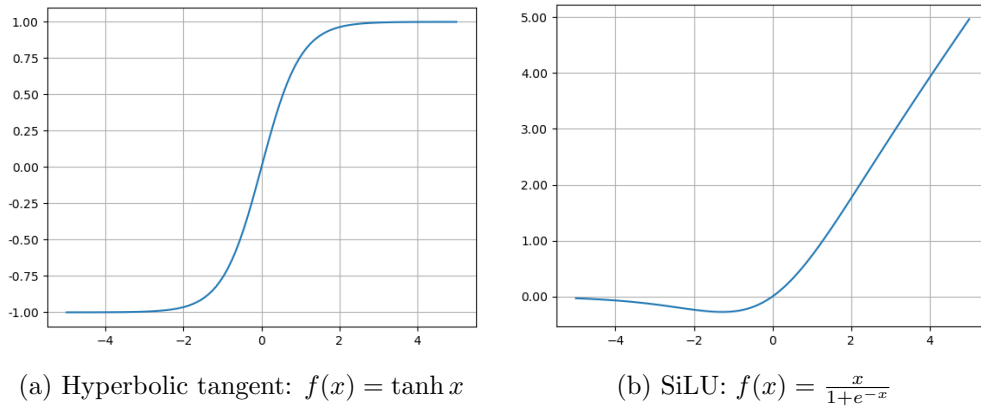


Figure 2: Two standard activation functions for the neurons of ANN.

2.2.2 Architecture: multiple neurons, layers and connections

Multiple single neurons can be connected to each other in a layered structure, as depicted in figure 3. Each neuron in the first layer will take 1 one input, so there will be as many neurons as inputs in $\{x_i\}$. In general, the neurons in the first layer do not have activation (they are just the “gates” to the NN model). Then, each will output a single value, which will be fed to all the other the neurons in the following layer as one of their inputs. Therefore, each neuron in an intermediate layer must have the same number of inputs as the number of neurons in the previous layer. An intermediate layer is called *hidden layer*. Finally, each neuron in the output

¹The first implementations of ANNs were done during the 1950s and 1960s. Frank Rosenblatt [2], considered one of the pioneers of deep learning, introduced *perceptrons*; artificial neurons that used a step function as activation. Perceptrons could only yield a binary output [1].

layer linearly combines the inputs (outputs of the last hidden layer) and activates them one last time. There are as many outputs as neurons in the output layer.

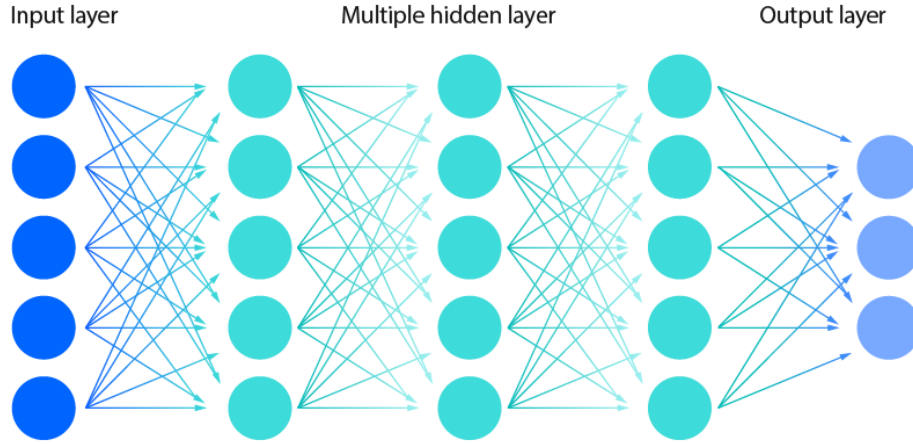


Figure 3: Deep (fully-connected) NN with 5 inputs, three hidden layers of 5 neurons each, and 3 outputs.

In the end, the output $F(x)$ of the NN will be a non-linear function coming from the composition of nested linear transformations $w_i x + b_i$ of the input x , “activated” by a non-linear function, $\text{activ}(w_i x + b_i)$. This function $F(x)$ will depend on *tunable-parameters* w_i and b_i . As we will see in the following section, the response of the NN will be controlled by these parameters. These will be tuned in a way that the output will satisfy the desired task or goal we “instruct” the NN to perform.

Below, we show for reference an explicit example of a NN with 1 input, 2 hidden layers of 2 neurons each, and 1 output, with a activation function $\tanh$ (Figs. 4 and 5)

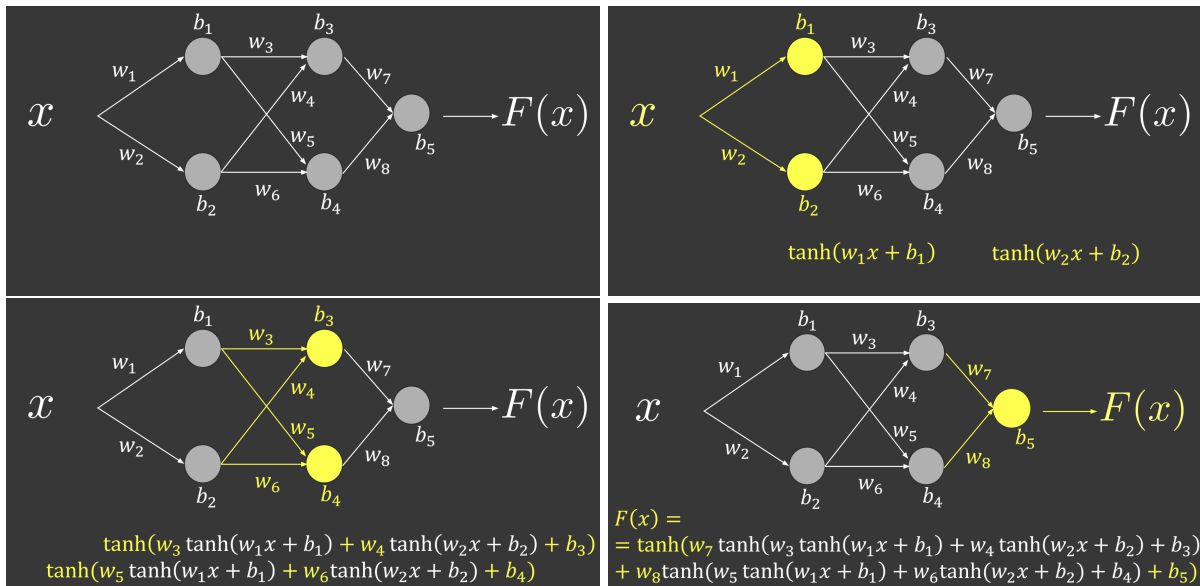


Figure 4: From top-left to bottom-right, response of each hidden layer to the input x . One can see how the response function $F(x)$ is a nested concatenation of the activation functions (in this case $\tanh$) applied on linear transformations $w_i x + b_i$.

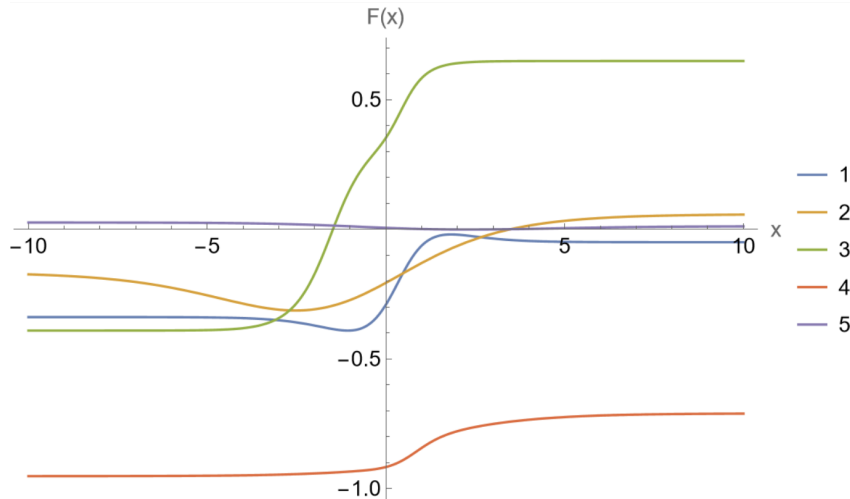


Figure 5: Five random initializations of the parameters (weights and biases) of the example NN in figure 4. Because it is very few layers and neurons, one can still see the structure of the activation function. However, it is clear that introducing many neurons and layers will give very non-linear responses $F(x)$.

The presence of at least one hidden layer with a smooth, non-linear activation function, such as the hyperbolic tangent, allows for a non-linear response of the NN. This fact makes it possible for the NN to replicate non-linear behaviors, and is at the root of the ability of NNs to be *universal approximators* (meaning that, in principle, they can approximate any function). This is extremely useful, and allows, for example, to solve non-linear differential equations (as described in Sec. 3).

2.3 Learning process

The last thing we need to introduce about NNs is the **process of learning**, which is usually split in two phases: the *training phase* and the *testing phase*.

2.3.1 Loss function: general case

The key to training a NN to solve a specific problem is to transform it into an optimization problem [3].

The most common mechanism is the **minimization of a loss function**. The loss function quantifies how well the output of the neural network model is performing compared to the task that it is instructed to do, e.g. image recognition, differential equation resolution, clustering, etc. Intuitively, the loss function is the way of “talking to the NN”, or “telling the NN what it is supposed to do”.

Depending on the type of task, the concrete form of the loss function will be one or other, but the idea behind it is always that one of finding a quantity to optimize (minimize or maximize).

Let us pick the example of image recognition. Later, in Sec. 3.2, we will define the loss function in the context of Physics Informed Neural Networks (PINNs) which allows to solve differential equations as an optimization problem.

Concretely, we now use as an example the task of recognizing handwritten digits[1]. Note that different examples of the same digit can have different forms based on size, calligraphy, thickness

of the lines, etc., as one can see in figure 6. Still, one would have no problem in recognizing that the same digit in different calligraphies represents in fact the same number.



Figure 6: Examples of handwritten digits from 0 to 9 (figure taken from [1])

As mentioned above, we first split the data in a *training set* (data that the NN model will *learn* from), and *testing set* (data that the NN model will use to check how well the training worked). This can be a 60 – 40% splitting, but it wildly depends on the nature, quantity, and quality of the dataset.

Imagine we now define a NN with the number of inputs being the number of pixels in each image, where each number that is inputted represents how “tinted” that pixel is (0 for completely white, and 1 for completely black). We choose a specific architecture for the hidden layers, and we choose 10 output neurons.

Heuristically, if the response is $(1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$, it means that the digit was recognized as a 0, $(0, 0.99, 0, 0, 0, 0, 0, 0.01, 0, 0)$ as a 1 with 99% probability and as a 7 with very low probability, $(0, 0, 0, 0, 0.5, 0, 0, 0, 0, 0.5)$ is “doubting” with 50% chance between 4 and 9, and $(0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1)$ is “completely unrecognized”.

In the training fashion relevant here, called **supervised training**, we will need to assign “labels” to the training and testing data images. This is necessary so that the untrained NN can correct itself, having a reference truth to converge to.

Now, the way to tell the NN that it should do this is through the loss function (the quantity that we will minimize). In many cases, we pick the mean square error (MSE) between the response of the NN and the underlying truth for each of the elements in the training data:

$$L_{\text{MSE}} = \frac{1}{n} \sum_x \left| y^{\text{NN}}(x) - y^{\text{label}}(x) \right|^2 \quad (2)$$

where the “tagged” output is $y^{\text{label}}(x = 0) = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$ for an input image with the handwritten digit 0, $y^{\text{label}}(x = 2) = (0, 0, 1, 0, 0, 0, 0, 0, 0, 0)$ for digit 2, and so on.

In this way, one can see that, if the response of the NN model is correct, i.e. it predicted an output that is the same as the “tag” associated to the input, the distance squared between vectors will be zero (they will be the same vector). For example, if the input is a handwritten 0:

$$y^{\text{label}}(x=0) = (1, 0, \dots, 0) \xrightarrow{\text{NN pred.}} \begin{cases} \text{If } 0 \rightarrow y^{\text{NN}}(x=0) = (1, 0, \dots, 0) \Rightarrow |y^{\text{NN}} - y^{\text{label}}|^2 = 0 \\ \text{If } 1 \rightarrow y^{\text{NN}}(x=0) = (0, 1, \dots, 0) \Rightarrow |y^{\text{NN}} - y^{\text{label}}|^2 = 1 \\ \text{If } 0 \text{ or } 9 \rightarrow y^{\text{NN}}(x=0) = (0.5, 0, \dots, 0.5) \Rightarrow |y^{\text{NN}} - y^{\text{label}}|^2 = 0.5 \end{cases}$$

One can clearly see that the loss function will be higher when the number of wrong predictions is higher, and it will keep on decreasing as the number of correct predictions increases. Therefore, minimizing the loss is equivalent to finding a response of the NN that performs as demanded.

2.3.2 Optimization process: Stochastic Gradient Descent (SGD)

The **minimization process** of the squared loss function is generally done through *stochastic gradient descent* techniques.

Gradient descent is based on the computation of the gradient of the squared loss function L for a given configuration of weights and biases $\{p_i\}$, in the m -dimensional phase space of parameters conformed by all the weights and biases (being m the number of parameters of the NN model):

$$\nabla_{p_i} L(p_i) \quad \forall \quad i = 1, \dots, m \quad (3)$$

Initially, the NN takes a random configuration p . The input data are passed through the NN, and gradient of the loss function with respect to the trainable parameters p_i is computed. Then, the NN’s configuration is changed as

$$p_i \rightarrow \hat{p}_i = p_i - \eta \nabla_{p_i} L, \quad (4)$$

where η is a small, positive parameter. In this way, the state p moves against the gradient of L . This process is repeated iteratively, so the state/-configuration of the NN “rolls down” towards the minimum of L (see figure 7).

Note the amount by which the NN moves in the loss space for each iteration is controlled by the parameter η . This parameter is called *learning rate*, and it needs to be carefully chosen. Larger learning rates allow the NN to flow towards a minimum more quickly, but have more chances of missing small or “thin” minima; the resolution decreases. On the other hand, if the learning rate is too small, the computation will be slow, and the NN could get “trapped” in a false minimum (local minimum, but not the global one). These issues concerning the learning rate are depicted in figure 8.

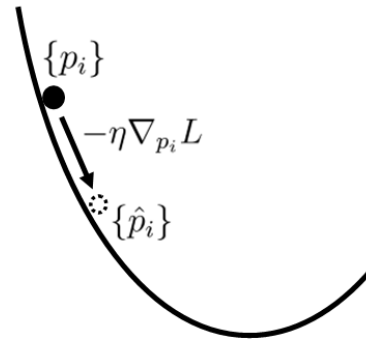


Figure 7: Gradient descent representation in hyperparameter space. From the state $\{p_i\}$, the NN moves opposite to the gradient of the loss function to the state $\{\hat{p}_i\}$.

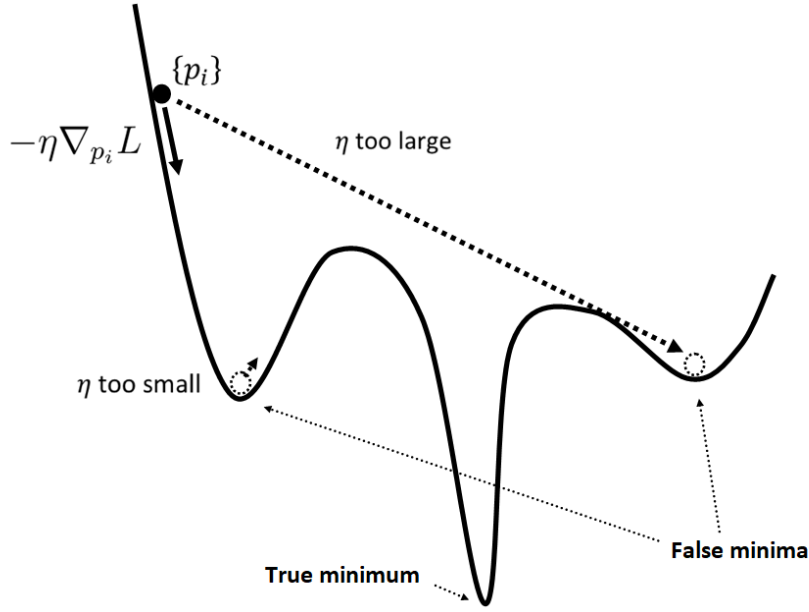


Figure 8: Visual representation on the importance of the choice of learning rate η . If η is too large, it might cause the NN to skip the true minimum. If η is too small, the NN might get stuck in a false minimum.

In reality, evaluating the gradient of the loss function in all the training points is costly (and might produce overfitting?), so one performs Stochastic Gradient Descent (SGD), where the gradient is computed and evaluated for a mini-batch of the whole training set, the weights are updated, and then it is computed for the next mini-batch, and so on, until the full training dataset is finished. This is considered as a single epochs (but it consists of many iterations).

2.3.3 Adam optimizer

We present here the hyperparameters of the **Adam** optimizer, which is a more elaborate version of SGD. The most relevant hyperparameters are:

- **Learning rate** η (as described in the previous section):

If the learning rate is too large, the state of the NN will jump around the minimum of the loss (in the space of parameters), never getting to the bottom. On the other hand, if the learning rate is too small and the state is in a region where gradients are small, it will get stuck. In the first case, we want the learning rate to be:

$$\eta \leq \frac{(x^* - x)}{\nabla_w L} \quad (5)$$

where x^* represents the position of the minimum of the loss in parameter space, and x represents the state of the NN in that same space.

- **Momenta** β_1 and β_2 :

The update on the weights is:

$$w_i^* = w_i - \frac{\eta^*}{1 + \sqrt{r^*}} \nu^* \quad (6)$$

where:

$$\nu^* = \beta_1 \nu + (1 - \beta_1)(\nabla_w L) \quad (7)$$

$$r^* = \beta_2 r + (1 - \beta_2)(\nabla_w L)^2 \quad (8)$$

Here, magnitudes with $*$ represent the updated values of the variable after one epoch of back-propagation.

The magnitude ν in equation (7) can be understood as a first momentum or inertia; even if the gradient $(\nabla_w L)$ is zero, there will be some “residual gradient” $\beta_1 \nu$ that will preserve the effective learning rate non-zero. If β_1 is large, the inertia will be large. This first momentum prevents the NN state to get stuck in **saddle points**, where the gradient is zero in all directions, but its not a minimum (see Fig. 9, left). This is very useful, since in a high dimensional space, the probability of flowing towards saddle points is extremely high².

The magnitude r controls how large are the components of a step taken, depending on the magnitude of the component of the gradient. In other words, if there are two curvatures that have very different scales (like a “semi-tube” shape), the gradient will be much larger in the direction across the tube, and smaller along the tube. β_2 controls the *increase* of the step taken in the direction *along* the tube (so in the direction of the smaller component of the gradient), and the *decrease* in the component of the step *across* the tube (see Fig. 9, right).

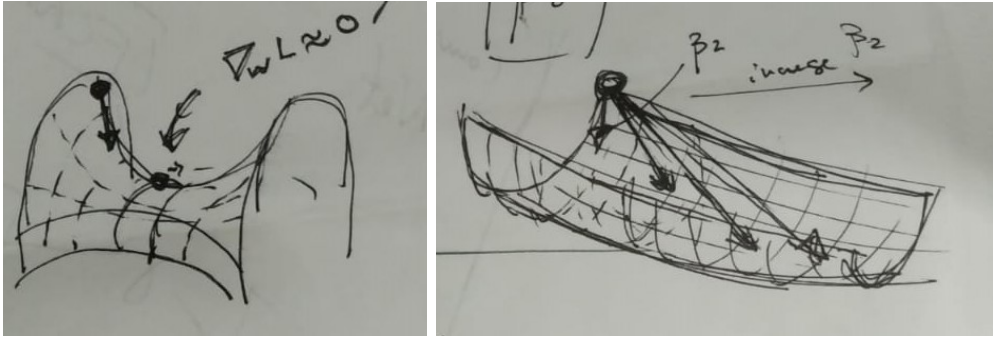


Figure 9: Sketch of the situations that the parameters β_1 (left) and β_2 (right) attempt to tackle during the training process.

2.3.4 Overfitting

3 PINNs

3.1 Definition

Physics Informed Neural Networks (PINNs) are a type of machine learning algorithm in which the Neural Networks themselves are used to get solutions of a particular differential equation (DE). In the setup that we are going to discuss, the NN itself will become the solution to a DE. As we have already explained, a large enough NN can be used as a “universal function

²If the parameter space was two dimensional, i.e. only two weights w_1, w_2 , there would be a 50-50% probability. If now we increase the dimensionality of the parameter space, the combinations of positive and negative curvature that give saddle points get larger than that of all positive curvature (minimum).

approximator” and, thus, can resemble any solution to a differential equation up to arbitrary precision.

Along this section we will explain PINNs in detail. First, we will start with the most important characteristic of this setup, the definition of the lost function. Then, we will discuss two different ways of implementing the initial and boundary conditions of a problem. To finish, we will explain how we can learn the dependence of the solutions on different parameters of boundary conditions of the differential equation in a so called “Bundle solution”. At last, we will comment on advantages and disadvantages of PINNs when compared to traditional numerical methods.

3.2 Loss function from the Differential Equations

As explained in section 2.3.1, the loss function is the tool that drives the neural network model to perform a specific task. PINNs aim to solve a differential equation.

To this aim, we start by defining the loss function of PINNs as the residual (squared) of the differential equation when evaluated at the predicted solution:

$$L_{\text{DE}} \propto \{\mathcal{D}[u^{\text{NN}}(x)] - f(x)\}^2 \quad (9)$$

where x is the independent variable and $u(x)$ is the function that is solution to the DE, given by the differential operator $\mathcal{D}$ and the function $f(x)$. The operator $\mathcal{D}$ encodes the set of derivatives that act on $u(x)$ to give the homogeneous part of the differential equation. $f(x)$ is the inhomogeneous part. For example, for a forced harmonic oscillator equation:

$$\frac{\partial^2 u(x)}{\partial x^2} + kx - \cos(x) = 0$$

the differential operator could be expressed as $\mathcal{D} = \partial_x^2$, and the forcing function is $f(x) = -kx + \cos(x)$.

Trivially, the exact solution $u(x)$ yields:

$$\mathcal{D}u(x) - f(x) = 0 \quad (10)$$

The goal of the PINN model will be to find $u^{\text{NN}}(x)$ such that the **DE residual is minimized** up to a certain accuracy. The residual R at each point x is:

$$\mathcal{D}u^{\text{NN}}(x) - f(x) \equiv R(x) \quad (11)$$

We will be sampling the independent variable in N discrete points. Further, if we are solving a system of differential equations, we will sum/average over the number of equations, and over the number of sampled points. The DE loss is then defined as:

$$L_{\text{DE}} = \sum_{i=1}^N \sum_j^{\# \text{ eqs}} R_j(x_i)^2 = \sum_{i=1}^N \sum_j^{\# \text{ eqs}} [\mathcal{D}u^{\text{NN}}(x_i) - f(x_i)]_j^2 \quad (12)$$

3.3 Initial or Boundary Conditions (IC/BC)

If one looks back at one of the differential equation courses one has taken, one may remember that, given a differential equation, boundary or initial conditions have to be specified so that the problem has a unique specific solution. So a fully defined ODE problem is specified by the DE, and the boundary or initial conditions:

$$\begin{cases} \mathcal{D}u(x) - f(x) = 0 \\ u(x_0) = u_0 \equiv \text{constant} \end{cases} \quad (13)$$

One important advantage on PINNs is that they can be trained simultaneously for many BC. Even more, a domain of K possible values for the BC can be sampled: $u_0, u_1, \dots, u_K \in [a, b]$, and the PINN can be trained on them. Doing the training in what is called a *bundle* (see Sec. 3.4), and once the training is finished, will allow to evaluate the model for any BC on the domain $[a, b]$ without the need to retrain.

Next, we describe two methods for imposing the BC or IC onto the solutions in the PINNs setup: a soft enforcing, and a hard enforcing.

3.3.1 Imposition on the loss function (soft enforcing)

The first approach is to add an additional loss function that penalizes the model if the learned solution $u^{\text{NN}}(x)$ does not satisfy the BCs/ICs. After convergence, the solution will satisfy the BCs to the precision given by this loss function.

$$u(x = x_0^b) = u_0 \quad (14)$$

where the superindex “ b ” indicates that x_0 is part of the domain’s boundary. Then:

$$L_{\text{BC}} \propto \left[u^{\text{NN}}(x_0^b) - u_0 \right]^2 \quad (15)$$

Additionally, if the boundary had more than one point, e.g. both sides of a 1D domain, or the perimeter of a 2D domain, we would need to sum over the number of points $\{x_i^b\}$ in the boundaries, N_b .

Then the total loss³ would be:

$$\begin{aligned} L_{\text{tot}} &= \lambda_{\text{DE}} L_{\text{DE}} + \lambda_{\text{BC}} L_{\text{BC}} \\ &= \lambda_{\text{DE}} \sum_{i=1}^N \sum_j^{\# \text{ eqs}} [\mathcal{D}u^{\text{NN}}(x_i) - f(x_i)]_j^2 + \lambda_{\text{BC}} \sum_{i=1}^{N_b} \sum_j^{\# \text{ eqs}} \left[u^{\text{NN}}(x_i^b) - u_0(x_i^b) \right]_j^2 \end{aligned} \quad (16)$$

Note that we have added two parameters λ_{DE} and λ_{BC} . These are relative weights given to each of the losses. Depending on the problem, these will be important to have a stable training. They can be selected and tuned by hand, or they can be set as trainable parameters to the model.

For example, if the problem at hand is such that $N \gg N_b$, it is possible that one would need $\lambda_{\text{BC}} > \lambda_{\text{DE}}$, so that the PINN “attends” both terms in the loss (note that since we are performing a mean over all points, if $\lambda_{\text{DE}} = \lambda_{\text{BC}}$, the contribution from the N_b points can be neglected with respect to the one coming from the N points).

³The notation in Eq. (16) is $u(x = x_i^b) = u_i$, being x_i^b one of the N_b points in the boundary of the domain, and u_i the value per point in the boundary (e.g. for a 1D domain with two fixed sides, one would have $u(x_0^b) = u_0$ and $u(x_1^b) = u_1$).

3.3.2 Imposition by construction in the architecture of the net (hard enforcing)

The second way of imposing the BCs is through a **reparameterization of the output** of the PINN model.

Specifically, if the output of the PINN is:

$$\tilde{u}^{\text{NN}}(x) = u_0 + \left(1 - e^{-\lambda(x-x_0^b)}\right) \cdot u^{\text{NN}}(x) \quad (17)$$

then one can see that $\tilde{u}(x = x_0^b) = u_0$ **by construction**⁴.

If there are 2 Dirichlet BCs:

$$\tilde{u}^{\text{NN}}(x) = u_0(1 - \tilde{x}^b) + u_1\tilde{x}^b + \left(1 - e^{-\lambda(x-\tilde{x}^b)\tilde{x}^b}\right) \cdot u^{\text{NN}}(x) \quad (18)$$

where:

$$\text{where } \tilde{x}^b = \frac{x - x_0^b}{x_1^b - x_0^b} \quad (19)$$

One can check that $\tilde{u}(x = x_0^b) = u_0$ and $\tilde{u}(x = x_1^b) = u_1$.

This way of imposing the BCs/ICs has the advantage that it does not require tuning of the relative loss weights λ_i (there is only the DE loss). This can also be very useful in problems where the training is unstable, or the exact satisfaction of the BCs is necessary.

This reparameterization might have the disadvantage that it lowers the expressability of the PINN model, since for values of x close to the BC value x_0^b , the pre-factor $(1 - \exp[-\lambda(x - x_0^b)])$ is small, so the model would need to learn unreasonably large values to yield an order 1 value of the product.

3.4 Bundle solutions

When working with DEs it is common to require multiple solutions corresponding to different initial and boundary values. In dynamical systems, each of these solutions represents a trajectory, tracing out an alternate time evolution of the state of the system. Similar situations apply when we have multiple boundary values. In addition, when an ODE depends on a parameter, for example, a physical constant whose value has an associated uncertainty, it can be useful to have different solutions for different values of the parameter.

Let us suppose that the ODE that we want to solve is the following

$$\begin{cases} \mathcal{D}_\nu u(x) - f(x) = 0 \\ u(x_0^b) = u_0 \end{cases} \quad (20)$$

where $u(x)$ is the unknown function, x the independent variable, ν is a parameter appearing on the equation, and u_0 the initial condition. In bundle solutions, we can train the PINN on N_k sets of BCs at the same time, each of which would give a different solution $u_k(x)$ with $u_k(x_0^b) = u_{0,k}$. Let us assume that we are imposing the BC to be exactly satisfied as demonstrated in the previous section 3.3.2. The same can be done for multiple values of some parameter ν . The key

⁴Of course, there are many possible parameterizations that will make the output satisfy BCs/ICs by construction. It is left as an exercise for the reader to find another one.

part in here is that now the response of the NN can be written as $u^{\text{NN}} = u^{\text{NN}}(x, \nu, u_{0,k})$, where we have included both the parameter ν and the different initial conditions $u_{0,k}$ as inputs of the model. Then, the loss function 12 can be easily generalized as:

$$L_{\text{DE}} = \sum_i^N \sum_j^{\# \text{ eqs}} \sum_k^{N_k} \sum_l^{N_\nu} R_j(x_i, \nu_l, u_{0,k})^2 = \sum_i^N \sum_j^{\# \text{ eqs}} \sum_k^{N_k} \sum_l^{N_\nu} [\mathcal{D}_{\nu_l} u^{\text{NN}}(x, \nu_l, u_{0,k}) - f(x)]_j^2 \quad (21)$$

where ν and $u_{k,0}$ now are sampled with a total number of values of N_ν and N_k respectively.

It is important to remark that this method is very powerful: we are letting the NN learn different solutions for different initial conditions and different values of ν . In this way, we can learn a large number of solutions (usually called *Bundle*) by training just once our model. In this way, generating a solution for a new initial condition is just trivial and can be done by just evaluating the model for that particular value of u_0 . This technique is also extremely valuable when dealing with inverse problem. In this kind of equations we want to recover a free form function just by using boundary data. This function is encoded on how the solutions change from one initial condition to another. In this way, learning this behavior will translate on learning the free form function in some sense.

3.5 Advantages and disadvantages

In this section, we compare the main advantages and disadvantages of Physics Informed Neural Networks (PINNs) with respect to traditional numerical methods. Both approaches aim to solve partial differential equations but differ fundamentally in how they compute derivatives, store data, and generalize across regimes. The following summary highlights these contrasts.

PINNs — Advantages

- Derivatives are computed exactly.
- Fast to evaluate.
- Only neural network parameters need to be stored.
- Parameters can be included as inputs (Bundle Solution).
- Transferable to other regimes (Multi-head setup).

Numerical Methods — Advantages

- Derivatives computed via finite differences.
- Evaluation is comparatively slow.
- The full grid with the solution must be stored.
- Must be recomputed for different parameters.
- Must be solved again for each configuration.

PINNs provide an efficient and flexible representation of physical systems. Their exact computation of derivatives via automatic differentiation enables high accuracy and fast evaluations once trained. They also require storing only the network weights, making them more memory-efficient than grid-based numerical schemes. Furthermore, their ability to incorporate physical parameters as inputs allows for generalization across different regimes. In contrast, classical numerical solvers depend on discretized grids, leading to slower evaluations and repeated computations for each parameter configuration.

PINNs — Disadvantages

- Slow to train.
- Hard to train on irregular domains.
- Difficult to tune.
- No formal error bound estimation.
- Poor generalization outside the training domain.

Numerical Methods — Disadvantages

- Much faster to compute.
- Boundary conditions easily satisfied on irregular domains.
- Guaranteed convergence.

Despite their flexibility, PINNs are challenging to train efficiently. Their optimization landscape is often stiff, requiring significant tuning and computational resources. Moreover, there are no formal error bounds or convergence guarantees, and performance deteriorates when extrapolating beyond the training domain. Numerical methods, while less adaptable, benefit from decades of theoretical development, offering well-defined convergence criteria and the ability to handle complex geometries with reliable precision.

4 Dimensionality reduction techniques

Dealing with data in science is inevitable. Imagine the physical situation in which we measure a certain number of parameters d across a number of samples n_s . In general, these d parameters will not be independent but there should exist a relation between them.

For example, let's suppose that we are measuring the absolute magnitude and the stellar mass across a wide number of different galaxies. When analyzing the data we would expect that these two parameters are not independent but that there exists a relationship between them: the larger the mass of the galaxy, the brighter it will be. Thus, when representing these samples they will not fill the entire plane spanned by luminosity vs mass, but they will cluster around “lower dimensionality structure”.

The goal of this section is to present different techniques to reduce the effective data dimensionality d to a lower number. For this purpose we will present different algorithms.

4.1 Principal component analysis

Principal Component Analysis (PCA) is an exploratory data analysis tool that aims to perform a **linear** dimensionality reduction on a data set. PCA will find the orthogonal projection vectors that account for the maximal amount of variance in the data. Imagine that we have a data matrix which contains number of samples n_s with d different properties:

$$X = \begin{pmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s \times d}, \quad \mathbf{x}_i \in \mathbb{R}^d.$$

The first step will be to center the data around the mean of every property measured. This can be done by applying the following expression

$$\bar{\mathbf{x}} = \frac{1}{n_s} \sum_{i=1}^{n_s} \mathbf{x}_i, \quad \tilde{X} = X - \mathbf{1}\bar{\mathbf{x}}^\top.$$

Once the data is centered, we can simply compute the covariance matrix as

$$C = \frac{1}{n_s - 1} \tilde{X}^\top \tilde{X}.$$

To get the principal direction along which the data is spanned we need to diagonalize the covariance matrix. The interpretation of this is easy, we want to get the directions along which the data has larger variance. The variance will be larger across a direction is along this direction we can approximately explain the data by a straight line. Having said that, we can compute the eigenvalues by solving the characteristic equation

$$C\mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d.$$

$$V_m = \begin{pmatrix} | & & | \\ \mathbf{v}_1 & \dots & \mathbf{v}_m \\ | & & | \end{pmatrix} \in \mathbb{R}^{d \times m}.$$

where we have ordered the eigenvalues by its value. Note that the eigenvector associated with the largest eigenvalue v_1 will tell us the direction along which the covariance is the largest. Note that for the eigenvectors we have only kept the most relevant m components, with $m \leq d$. These eigenvectors will be associated with the largest m eigenvalues. Finally, to visualize our data along these “reduced” directions we need to project them from $\mathbb{R}^d$ to $\mathbb{R}^m$. This can be easy done by applying the following expression

$$Z = \tilde{X}V_m, \quad \mathbf{z}_i = V_m^\top (\mathbf{x}_i - \bar{\mathbf{x}}).$$

where Z is now the “reduced” data matrix. Note that we have gone from d to m with $m \leq d$.

Let’s discuss some intuition about the PCA. If the geometry of the data when visualized $\mathbb{R}^d$ is a hyper-sphere, then we will get that all the eigenvalues of the covariance matrix are the same. However, if we deform this hyper-sphere into a hyper-ellipsoid, we will get that the largest eigenvalue will be the one associated to the eigenvector that points along the longest axis of the hyper-ellipsoid.

4.2 t-distributed stochastic neighbor embedding

T-distributed stochastic neighbor embedding, or t-SNE for friends, is a **non-linear**, probabilistic dimensionality reduction technique. In contrast to PCA, this technique aims to preserve the local structure of our data. This means that points that are close on the d dimensional space will also remain close when we reduce the dimensionality to m . This technique is normally used to embed high dimensional data to a lower dimensional space which typically has two or three dimensions. We will now discuss the particular mathematical formulation of t-SNE.

We will consider the same data that we had for the PCA. This data can be rearranged on a matrix X . This will consist on n_s samples with d properties each.

$$X = \begin{pmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s \times d}, \quad \mathbf{x}_i \in \mathbb{R}^d.$$

The goal of t-SNE will be to reduce this matrix X to the matrix Y , that can be written in the following form

$$Y = \begin{pmatrix} - & \mathbf{y}_1^\top & - \\ - & \mathbf{y}_2^\top & - \\ & \vdots & \\ - & \mathbf{y}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s} \times m, \quad \mathbf{y}_i \in \mathbb{R}^m.$$

where $m \leq d$. As we have already mentioned, we will typically consider this parameter to be 2 or 3. The first step would be to compute the condition probability $p_{j|i}$ which measures how likely it is for a point $\mathbf{x}_i$ to pick $\mathbf{x}_j$ as its neighbor. This can be quantified with the following expression

$$p_{j|i} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_k\|^2}{2\sigma_i^2}\right)}, \quad p_{ij} = \frac{p_{i|j} + p_{j|i}}{2n_s}.$$

where in the last expression we have symmetrized the probability, since being neighbors is a reciprocal relation. In the probability expression the σ_i are fixed from the very beginning such that they give the perplexity indicated by the user according to the following expression

$$H(P_i) = - \sum_{j \neq i} p_{j|i} \log_2 p_{j|i}, \quad \text{Perp}(P_i) = 2^{H(P_i)}.$$

for our purposes we will just think about this as a parameter that we input to the model. The second step of t-SNE is to compute the analogous of the p_{ij} but for the low-dimensional points $\mathbf{y}_i$. This probability is defined using a t-Student distribution in the following way

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}.$$

At last, we want both distributions to be as similar as possible. This is done by minimizing the so called Kullback-Leibler divergence $C(Y)$. This quantity is minimized by gradient descent with respect to the vectors $\mathbf{y}_i$. We also show the derivative along which the $\mathbf{y}_i$ vectors are optimized.

$$C(Y) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad \frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij}) (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1} (\mathbf{y}_i - \mathbf{y}_j).$$

4.3 Uniform manifold approximation and projection

The uniform manifold approximation and projection (UMAP) is often presented as an alternative to t-SNE, since it is also a non-linear dimensionality reduction technique. The main difference with respect to t-SNE is that it will try to preserve both local and global structure of the data manifold. Its implementation is based on approximating the fuzzy topological structure of the data manifold. We will now explain the particular mathematical implementation of this method.

As we have already done, we will start our data matrix X that we want to reduce to Y , both of them described as follows:

$$X = \begin{pmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s} \times d, \quad \mathbf{x}_i \in \mathbb{R}^d.$$

$$Y = \begin{pmatrix} - & \mathbf{y}_1^\top & - \\ - & \mathbf{y}_2^\top & - \\ & \vdots & \\ - & \mathbf{y}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s \times m}, \quad \mathbf{y}_i \in \mathbb{R}^m.$$

The key element regarding the UMAP computation will be the so-called weighted k-nearest neighbor (kNN) graph. For each data point $\mathbf{x}_i$ the kNN will compute the distances to its k nearest neighbors. Then, two necessary quantities are computed. The first one is called local connectivity threshold ρ_i , and it is defined as the distance to the closest non-identical point on the data manifold. Once ρ_i is computed we need to find σ_i which will enter in the expression to define the **local fuzzy membership strengths** $w_{j|i}$:

$$w_{j|i} = \exp\left(-\frac{\max(0, d_{ij} - \rho_i)}{\sigma_i}\right).$$

These σ_i 's are defined such that the sum of these parameters is normalized to

$$\sum_j \exp\left(-\frac{\max(0, d_{ij} - \rho_i)}{\sigma_i}\right) = \log_2(k)$$

where k is the number of local neighbors. This is a parameter that we choose at the very beginning of the pipeline. Once these two parameters are found, the local fuzzy membership strengths are symmetrized as follows

$$w_{ij} = w_{i|j} + w_{j|i} - w_{i|j}w_{j|i}.$$

This same game is then played with the connection of the embedding Y . These connection strengths are defined with the following expression:

$$\tilde{w}_{ij} = \frac{1}{1 + a\|\mathbf{y}_i - \mathbf{y}_j\|^{2b}},$$

where a and b are hyperparameters that are internally computed by UMAP. At last, the embedding Y is optimized to make $\tilde{w}_{ij}$ similar to w_{ij} . This can be achieved by optimizing the following binary-cross entropy loss function via stochastic gradient descent:

$$C(Y) = - \sum_{i < j} (w_{ij} \log(\tilde{w}_{ij}) + (1 - w_{ij}) \log(1 - \tilde{w}_{ij})).$$

Once the optimization is done, we will expect an embedding Y that preserves the connectivity of the original data set. This will maintain the local structure (nearest neighbors), but also the medium scale structure (connectivity between neighbors).

4.4 Machine learning approach: autoencoders

The last two techniques that we have presented consist of optimizing some loss function. However, the form of this function is very concrete depending on the method. The autoencoders approach tries to use all the machine learning power to perform a dimensionality reduction on a data set. We will now explain the details on how autoencoders work

In general, an autoencoder is a Neural Network that will map our data vectors to themselves. This is, imagine that we have the same data matrix as before

$$X = \begin{pmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_{n_s}^\top & - \end{pmatrix} \in \mathbb{R}^{n_s} \times d, \quad \mathbf{x}_i \in \mathbb{R}^d.$$

Then, the autoencoder will map each of the vectors to $\tilde{\mathbf{x}}_i$ to them selves

$$\mathbf{x}_i \longrightarrow \tilde{\mathbf{x}}_i \simeq \mathbf{x}_i,$$

the key part is that the machine learning algorithm will do this while compressing the data to a low dimensional space $m < d$. We will now explain the parts that compose the autoencoder.

An autoencoder is made up of two different parts: the encoder and the decoder. The encoder will map, in a very non-linear way, the data vector into a vector in a lower dimensional space $m < d$. The decoder will map this vector to the original dimension $d > m$. In general, we can write the action of this two parts as follows

$$\begin{aligned} \text{Encoder:} \quad & E_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad \mathbf{z}_i = E_\theta(\mathbf{x}_i) \\ \text{Decoder:} \quad & D_\phi : \mathbb{R}^m \rightarrow \mathbb{R}^d, \quad \tilde{\mathbf{x}}_i = D_\phi(\mathbf{z}_i). \end{aligned}$$

In here D_ϕ and E_θ represent very non-linear function constructed by concatenations of different layers of made up of neurons. These two structures will possess a set of tunable weights and biases (represented by ϕ and θ) that we can vary to select the global response of the autoencoder. In here, the lower dimensional vector $\mathbf{z}_i$ is called the embedding/latent space of the autoencoder, and will be the reduced data we want to get out of the original vectors $\mathbf{x}_i$.

On every machine learning algorithm we have to define a loss function. In our setup the most natural way will be to define the loss as the reconstruction error when going from $\mathbf{x}_i$ to $\tilde{\mathbf{x}}_i$. Formally, this can be written as

$$\mathcal{L}(\theta, \phi) = \frac{1}{n_s} \sum_{i=1}^{n_s} \|\mathbf{x}_i - g_\phi(f_\theta(\mathbf{x}_i))\|^2$$

and then θ and ϕ are found by optimizing $\mathcal{L}$ using stochastic gradient descent. As a result of the training, the encoder will map our original data $\mathbf{x}_i$ to a reduced latent space $\mathbf{z}_i$. It is worth remarking here that if we use just one linear layer for the encoder and decoder, then the autoencoder technique will reduce to the PCA technique up to normalization and orthogonality conditions.

5 Hands-on sessions

There are two hands-on python sessions.

5.1 Solving the Forced Harmonic Oscillator (FHO) and the Van der Pol Oscillator (VdPO) in a bundle using PINNs

For this hands-on session, check the Interactive Python Notebooks in https://github.com/ptejerina/P2_ML_Lectures_public/tree/main

5.2 Dimensionality reduction with NNs

References

- [1] Michael A. Nielsen. Neural networks and deep learning. *Determination Press*, 2015.
- [2] Frank Rosenblatt. Perceptron simulation experiments. *Proceedings of the IRE*, 48(3):301–309, 1960.
- [3] NeuroDiffEq Developers. Neurodiffeq documentation, 2019. Available at <https://neurodiffeq.readthedocs.io/en/latest/index.html>.